SWEEP rule on the KD-Toric code

February 9, 2026

1 SWEEP Rule Decoder on the Toric

The decoder works as follows: each vertex v looks for a suggestion $\hat{v}$ on $\uparrow(v) \cap lk_2$ that matches σ_v^+. If there are more than one, then pick the one which has the minimal weight.

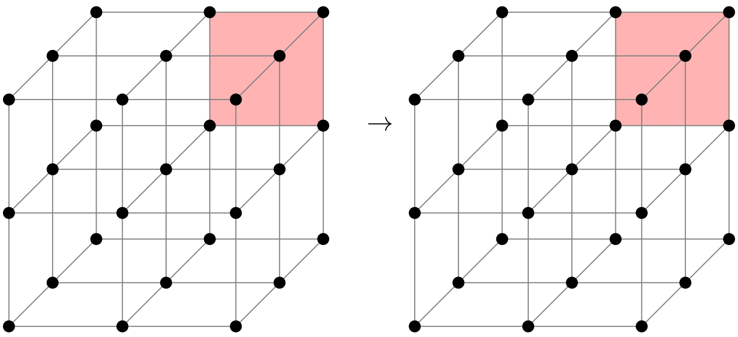

Figure 1: SWEEP rule

2 Span Expansion

Define the potentials functions $L_i : \mathbb{R}^d \to \mathbb{R}$ as follow[1]:

$$L_i(\mathbf{x}) = \begin{cases} \mathbf{x}_i & i < d+1 \\ -\langle \mathbf{1}, \mathbf{x} \rangle & i = d+1 \end{cases}$$

The **Size** of a subset S is defined as:

$$\mathbf{Size}(S) = \sum_i \max_{\mathbf{x} \in S} L_i(\mathbf{x})$$

Numerically, we show that the span property holds for the checks. Namely, let e be an unsatisfied check at time t, and let H_e be the unsatisfied checks at time $t-1$. Then there is a set $H_e' \subset H_e$ such that:

1. For any $i < d+1$ there exists $e' \in H_e'$ such $L_i(e') \geq L_i(e)$.
2. There exists $e' \in H_e'$ such that $L_{d+1}(e') \leq L_{d+1}(e) + 1$.

Proof. Consider a vertex v which at time t adjoins to an unsatisfied edge e, let u, w be vertices which have a positive faces that adjoin to e.

[1] Notice that we use opposite signs relative to Berman and Peter Gacs, yet we agree with Perskil.

1. If $\hat{u}|_e \neq \emptyset$, then the parallel edge to e going out from u belongs to H_e. Denote it by e_u, taking e_u satisfies at least two (three in 4D) of the first d inequalities and the last $d+1$th inequality. Denote by $x \in [d]$ the index of the inequality which isn't satisfied by e_u. So it remains to prove the existence of $e' \in H_e$ for which $L_x(e') \geq L_x(e)$.

   Consider the assignment of bits at time $t-1$ over the complex, and consider the graph $G = (\mathcal{F}, E)$ induced by associating vertices with any face that is set to 1 by the assignment. Two vertices are connected if their origin faces share an edge. Let $K \subset \mathcal{F}$ be a connected component of faces that contains the faces supported on u.

   We say that K is d far from u if:

   $$\max_{f \in K} |f_x - u_x| \leq d$$

   Now, we will prove by induction on d that there is a face $f \in K$ with $f_x \geq v_x$ such one of its edge is not satisfied.

   (a) **Base.** The edge from v which point at the $\hat{z}$ direction must not be satisfied.

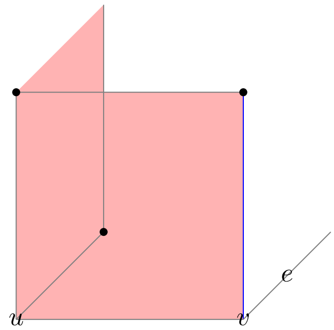

Figure 2: The induction base: In red, we have the faces on the support of u. The blue edge has the same x-coordinate as e. Any attempt to zero out the syndrome on the edge results in adding a face that is at least 2-far from u. Any configuration which is at most 1-far from u has to have that local view, yet might be connected to more faces which don't adjoin the blue edge.

   (b) **Assumption.** Assume that for any K which is at most $d-1$ far from u there is a face, with adjoins edge that is both has a x-coordinate that is greater than u_x and is not satisfied.

   (c) **Induction Step.** Consider a connected component K that is d-far from u. Notice that if by substitute a co-boundary (stabilizer) from K we get K' that is at most d' far from u for some $d' < d$ then we done. That because: $\partial_2 K' = \partial_2(K+z) = \partial_2 K$ for some $z \in \ker \partial_2$.

   On the other hand, **[COMMENT]** Finish that.

2. Else, if for any $u \in \downarrow(u) \cap lk_1$ it holds that $\hat{u}|_e = \emptyset$, then it must be that $e \in H_e$ because $\hat{v}$ can only decrease the syndrome on v. Moreover, it follows that $\hat{v} = \emptyset$ (otherwise, the suggestion would zero out the syndrome on $\uparrow(v) \cap lk_2$). Thus, $\sigma_v \not\subset \uparrow(v) \cap lk_2 \Rightarrow$ there is also an edge $e' \in H_e$ where $e' \in \downarrow(v)$. In that case, take $e \in H_e$ as the edge satisfies the first d equalities and e' for satisfying the $d+1$th inequality.

□

3 Investigation

Definition 3.1 (Investigation)**.** For any unsatisfied check e at time t, we define V (the set of checks which participated in 'misleading' of e), and two sets of edges on V, Arrows and Forks via the following algorithm:

let $V = \{e\}$, Arrows $= \emptyset$, Forks $= \emptyset$
let $Q \leftarrow$ queue $\{e\}$
while Q *is not empty* **do**
  $e' \leftarrow$ dequeue Q
  if $e' \notin$ *Error* **then**
    $e_1, .., e_l \leftarrow$ Excuse(e')
    Insert $\{e_i, e'\}$ to Arrows
    Insert $\{e_i, e_j\}$ to Forks
  end
end

Algorithm 1: Investigation